CRYPTOGRAPHY AND DATA SECURITY

PROJECT REPORT

RSN (Root-based Substitution and Number Theory) Cipher Algorithm

1. Introduction:

Cryptography means the method of securing sensitive information through the technique of transforming plaintext into an incomprehensible form called ciphertext. This field provides important means for protecting information and ensuring secure information exchange, data, and message transmission. Classical ciphers like Caesar Cipher, Vigenere, Playfair Cipher, Hill Cipher, Rail Fence Cipher, Route Cipher, Columnar Transposition Cipher, and Disrupted Transposition Cipher are important cryptographic concepts that include substitution, shifting, matrix operations, and permutation. However, most of the classical ciphers are insecure on their own due to the fact that they could be easily decrypted using frequency analysis or other attacks.

In this project, we propose a new symmetric cryptographic algorithm, known as RSN Algorithm. Our algorithm uses techniques of classical cipher methods and some fundamental principles of number theory, including Prime Numbers, Modular Arithmetic, Greatest Common Divisor (GCD), Euler's Totient Function, Relative Primes, Primitive Roots, and Modular Inverse.

It is a symmetric block cipher algorithm, where the secret key is shared for both encryption and decryption operations. This calls for another separate phase in our project that deals with sharing of the secret key between two parties using modular exponentiation and mathematical locking.

2. Steps of the RSN Encryption Algorithm:

Step 1: Plaintext Preparation

The message is prepared before being encrypted. First, all letters are transformed to the upper case to make sure that the algorithm operates with the fixed set of characters. Then, all unsupported characters are replaced or padded if the length of the message does not fit completely in one block.

Step 2: Character-to-Number Conversion

Each character is converted into a number. The algorithm uses 29 supported symbols: the 26 English uppercase letters, space, full stop, and question mark.

For example, A is represented as 0, B as 1,…Z as 25. This step is required because encryption calculations are performed on numbers, not directly on letters.

Step 3: Block Formation

After converting the message into numbers, the values are packed into one large number. This way, multiple characters are encoded in one big number, making the cryptanalysis much more difficult since ciphertext does not contain letters anymore.

Step 4: Block Splitting

Each complete block is divided into two equal parts, a left half and a right half. These two halves are used in the Feistel structure of the algorithm. Feistel algorithm allows one half of the block to be transformed and then mixed with the other half during each round.

Step 5: Round Key Generation

The secret key is converted into numeric form and then multiplied many times to get the different keys to use in the process of the next rounds. Thus, every round uses its own key rather than just one repeated many times.

Step 6: Round Function Processing

In each round, the right half of the block is passed through a round function. This round function performs several operations.

First, the right half is mixed with the round key. Then, a primitive-root-based transformation is applied to make the output less predictable. After that, the large number is split into smaller parts so that matrix multiplication can be performed. The values are then mixed using a key-generated matrix and rearranged using a key-based permutation.

This step is the main part of the algorithm.

Step 7: Feistel Combination

After the round function produces its output, that output is combined with the left half using modular arithmetic. Then the two halves are swapped. This Feistel structure is important because it allows the algorithm to be reversed during decryption, even though the round function itself is complex.

Step 8: Multiple Round Repetition

The same process is repeated for several rounds. The proposed RSN Algorithm uses 16 rounds. Using multiple rounds increases security because each round adds more confusion and diffusion. A small change in the plaintext or key can spread across the ciphertext after several rounds.

Step 9: Ciphertext Generation

After all rounds are completed, the final left and right halves are joined together. The result is the ciphertext. Since the encrypted values are large numbers, the ciphertext can be represented in hexadecimal form to make it easier to store, copy, and transmit.

3. Steps of the Key Sharing Process:

Step 1: Public Setup

Alice and Bob first agree on a large public prime number. This number is not secret and can be known by anyone. It is used as the main mathematical base for the key sharing process.

Step 2: Session Key Selection

Alice creates a random session key. This session key will later become the master key for the RSN Algorithm. The session key is not sent directly to Bob because sending it openly would allow an attacker to capture it.

Step 3: Alice Applies Her Lock

Alice chooses her own private locking value and uses it to lock the session key mathematically. She also calculates a matching unlocking value, which only she knows. Alice then sends the locked version of the session key to Bob. If an attacker captures this value, they only see the locked version, not the actual session key.

Step 4: Bob Applies His Lock

Bob receives Alice's locked value and adds his own private lock to it. Now the session key is protected by both Alice's lock and Bob's lock. Bob sends this double locked value back to Alice. Even if an attacker captures it, the attacker cannot recover the original session key because both locks are still applied.

Step 5: Alice Removes Her Lock

Alice receives the double locked value from Bob and removes only her own lock. After this step, the value is still protected by Bob's lock. Alice sends this Bob-locked value back to Bob.

Step 6: Bob Removes His Lock

Bob receives the value that is now protected only by his own lock. He removes his lock and recovers the original session key. Now both Alice and Bob have the same session key, but the session key was never sent directly during transmission.

Step 7: Using the Shared Key

The recovered session key is used as the master key for the RSN Algorithm. Alice uses it to encrypt the message, and Bob uses the same key to decrypt the message. Because both users share the same key, the algorithm remains symmetric.

4. Mathematical Solution:

4.1 Character Set:

The RSN Algorithm uses the following 29-symbol character set:

A-Z, SPACE, ., ?

The mapping is:

A	0
B	1
C	2
D	3
E	4
F	5
G	6
H	7
I	8
J	9
K	10
L	11
M	12
N	13
O	14
P	15
Q	16
R	17
S	18
T	19
U	20
V	21
W	22
X	23
Y	24
Z	25
SPACE	26
.	27
?	28

4.2 Plaintext Example:

Select a large Mersenne Prime Number P
$P = 2^{61} - 1$
P = 2305843009213693951

Select g which should be a primitive root to P
g = 37

Plaintext: HELLO WORLD?CRYPTO TEST.

Secret key: SECURITY

The plaintext is divided into two 12-character halves:

Left half = HELLO WORLD?

Right half = CRYPTO TEST.

4.3 Base-29 Multiplication:

$X = c1 \times 29^{11} + c2 \times 29^{10} + c3 \times 29^{9} + \ldots + c12 \times 29^{0}$
where c is the character in the separated halves

L0 = 87251736034664600

R0 = 31909054319148196

4.4 Seed Calculation:

The secret key SECURITY is converted into numbers:

S	E	C	U	R	I	T	Y
18	4	2	20	17	8	19	24

Now calculate the Seed = $\Sigma((i + 1) \times key\_i)$

Seed = 1×18 + 2×4 + 3×2 + 4×20 + 5×17 + 6×8 + 7×19 + 8×24

Seed = 570

The seed generated from the key is: 570

4.5 Round 1 Key Generation:

The Round 1 key is generated using the seed, primitive root value, and large prime modulus.

$K1 = g^{(seed+1)} \bmod P$
$K1 = 37^{571} \bmod 2305843009213693951$
$K1 = 393207995729065803$

4.6 Round 1 Processing

The right half is first mixed with the Round 1 key:

$T = (R0 + K1) \bmod P$
$T = (31909054319148196 + 393207995729065803) \bmod 2305843009213693951$
$T = 425117050048213999$

Then primitive root substitution is applied:

$U = g^T \bmod P$
$U = 37^{425117050048213999} \bmod 2305843009213693951$
$U = 17904715021354397$

The value U is split into four smaller values using base 65537:
$17904715021354397 = 63 \times 655373^3 + 39808 \times 655372^2 + 6889 \times 65537^1 + 20213 \times 65537^0$

This way, we can convert the large number into a vector for matrix mixing: [63, 39808, 6889, 20213]

A key-generated Round 1 matrix is applied:

M1 =

[34964 23893 40260 55999]

[56716 25908 134 29537]

[11533 25313 60035 25809]

[4671 9002 62754 28110]

The matrix is generated differently for every round using the formula:

$M_r[i][j] = Seed + 37r + key[(i + j) \bmod key\_length] \times (i + 1) + key[(4i + j) \bmod key\_length] \times (j + 1) + 29(i + 1)(j + 1) \bmod 65537$

The determinant condition is checked:

$\det(M1) \bmod 65537 = 54429$

Since the determinant is not zero, the matrix is valid.

After matrix multiplication, the vector = [50890, 18924, 14106, 39319]

A permutation key is generated from the secret key using the formula:

score_i = (key[(round + i) mod key_length] + 4i) mod 29

This formula is used 4 times, from position 0 to position 3. The resulting scores are sorted from smallest to largest which decides the permutation key for each round.

The permutation key for Round 1 is = [0, 3, 1, 2]

After permutation:

[50890, 39319, 18924, 14106]

Repack the values $V = 508890 \times 655373^3 + 39319 \times 655372^2 + 18924 \times 65537^1 + 14106 \times 65537^0$

After repacking, this gives the final output of the round function which is going to be used in the Feistel round:
F(R0, K1) = 490028112398639469

Now the Feistel round is applied:

L0 = 87251736034664600
R0 = 31909054319148196
F(R0, K1) = 490028112398639469

L1 = R0
L1 = 31909054319148196

R1 = (L0 + F(R0, K1)) mod P
R1 = (87251736034664600 + 490028112398639469) mod 2305843009213693951
R1 = 577279848433304069

This completes Round 1.

The same process is repeated for 16 rounds.

After 16 rounds, the final ciphertext for this example is:

L16 = 1799769367515085762

R16 = 877783770123441019

In hexadecimal form: 18fa10b4445dbbc2-0c2e83b02fcf937b

4.7 Key Sharing

For easy understanding, a small prime is used in this example. In the real algorithm, a very large prime is used.

Let: $P = 17$, $\varphi(P) = 16$

Alice wants to share the session key:
$S = 7$

Alice chooses her private lock:
$a = 3$

Since $\gcd(3, 16) = 1$

Now Alice needs the inverse of 3 mod 16. That means she needs a number that gives remainder 1 when multiplied by 3:
$3 \times 11 = 33$
$33 \bmod 16 = 1$

So, Alice can calculate her unlocking value:
$a^{-1} = 11$

Bob chooses his private lock:
$b = 5$

Since $\gcd(5, 16) = 1$

Now Bob finds the inverse of 5 mod 16
$5 \times 13 = 65$
$65 \bmod 16 = 1$

So, Bob can calculate his unlocking value:
$b^{-1} = 13$

Alice locks the session key:
$X = S^a \bmod P$
$X = 7^3 \bmod 17 = 3$

Bob adds his lock:
$Y = X^b \bmod P$
$Y = 3^5 \bmod 17 = 5$

Alice removes her lock:
$Z = Y^{\wedge(a-1)} \bmod P$
$Z = 5^{11} \bmod 17 = 11$

Bob removes his lock:

$S = Z^{\wedge(b-1)} \bmod P$
$S = 11^{13} \bmod 17 = 7$

Bob successfully recovers the original session key: S = 7

This shared key can now be used as the Seed for the RSN Algorithm.

5. Limitations:

5.1 Limited character support: The algorithm supports only uppercase letters, space, full stop, and question mark. Numbers and special characters must be replaced or normalized before encryption.

5.2 Fixed block size: The algorithm works on fixed-size plaintext blocks. If the message is shorter, padding is required. If it is longer, it must be divided into multiple blocks.

5.3 Ciphertext can become lengthy: Since plaintext is converted into large numeric blocks, the final ciphertext may be longer than the original message.

5.4 Complex implementation: The algorithm involves several steps such as packing, matrix generation, permutation, modular arithmetic, and key sharing, so implementation mistakes can affect encryption or decryption.

5.5 Padding handling is required: The receiver must know how padding was added so it can be removed correctly after decryption.

5.6 Depends heavily on key security: If the session key or private locking values are exposed, the security of the encryption process is weakened.

6. Implementation:

```
==========================================================
RSN Algorithm Implementation
Symmetric Encryption with Key Sharing
Made By Nehan Shah
==========================================================

Enter plaintext: Cryptography and Data Security

Is the channel secure for sharing the secret key? (yes/no): no

================ KEY SHARING PROCESS ================

The channel is not secure, so RSN will use the key-sharing process.

Public values:

P = 2305843009213693951

Euler Totient φ(P) = P - 1 = 2305843009213693950

Enter numeric session key S or press Enter to generate randomly: 891

Alice selects session key:

S = 891

Alice chooses private lock:

a = 2093909277736013887

Alice's unlock value a_inverse = 600700366380348973

Step 1: Alice locks the session key

Alice sends X = S^a mod P

X = 148135395713211014

Bob chooses private lock:

b = 351128134229756731

Bob's unlock value b_inverse = 2134601155866464671

Step 2: Bob adds his lock

Bob sends Y = X^b mod P

Y = 1653765459223763758
```

Step 3: Alice removes her lock

Alice sends Z = Y^a_inverse mod P

Z = 1944773982956364843

Step 4: Bob removes his lock

Bob recovers S = Z^b_inverse mod P

Recovered S = 891

Key sharing successful.

Alice and Bob now share the same session key.

This recovered S will now be used as the RSN master seed.

==========

========== PLAINTEXT PREPARATION ==========

Prepared plaintext: CRYPTOGRAPHY AND DATA SECURITY??????????????????

Original prepared length before padding: 30

==========

Showing Round 1 and Round 2 details for the first plaintext block only.

========== ROUND 1 DETAILS ==========

Left input: 31909053907410224

Right input: 317403797839991462

Round key: 1460563932527499952

After key mixing, T: 1777967730367491414

After primitive-root substitution, U: 1729139274692696139

U split into 4 values using base 65537: [6142, 56070, 19764, 55315]

Round matrix:

[28163, 50804, 61112, 57493]

[62829, 39078, 12272, 3526]

[32188, 3111,   57784, 16259]

[28143, 913,   1037,   5618]

Determinant mod 65537: 6524

After matrix diffusion: [52606, 12750, 10928, 6153]

Key-based permutation: [0, 1, 3, 2]

After permutation: [52606, 12750, 6153, 10928]

Round function output F: 972947165957603851

New left: 317403797839991462

New right: 1004856219865014075

==========

========== ROUND 2 DETAILS ==========

Left input: 317403797839991462

Right input: 1004856219865014075

Round key: 1006476291602537351

After key mixing, T: 2011332511467551426

After primitive-root substitution, U: 1106487503886660101

U split into 4 values using base 65537: [3930, 55925, 54936, 61854]

Round matrix:

[28868, 60496, 51080, 63117]

[8790, 13931, 15730, 41335]

[46644, 5501, 23312, 51542]

[6488, 59859, 19297, 42254]

Determinant mod 65537: 54961

After matrix diffusion: [61921, 37601, 57696, 56472]

Key-based permutation: [2, 0, 3, 1]

After permutation: [57696, 61921, 56472, 37601]

Round function output F: 100088570932183745

New left: 1004856219865014075

New right: 417492368772175207

==========

========== FINAL ENCRYPTED TEXT ==========

0b8db0b612d9e9c9-103f7a963699a85f | 1b07bfb325738811-182618ae44b8090e

==========

========== DECRYPTION CHECK ==========

Decrypted text: CRYPTOGRAPHY AND DATA SECURITY

==========